



TBWB

Angular Advanced Internationalization – i18n



Peter Kassenaar
info@kassenaar.com



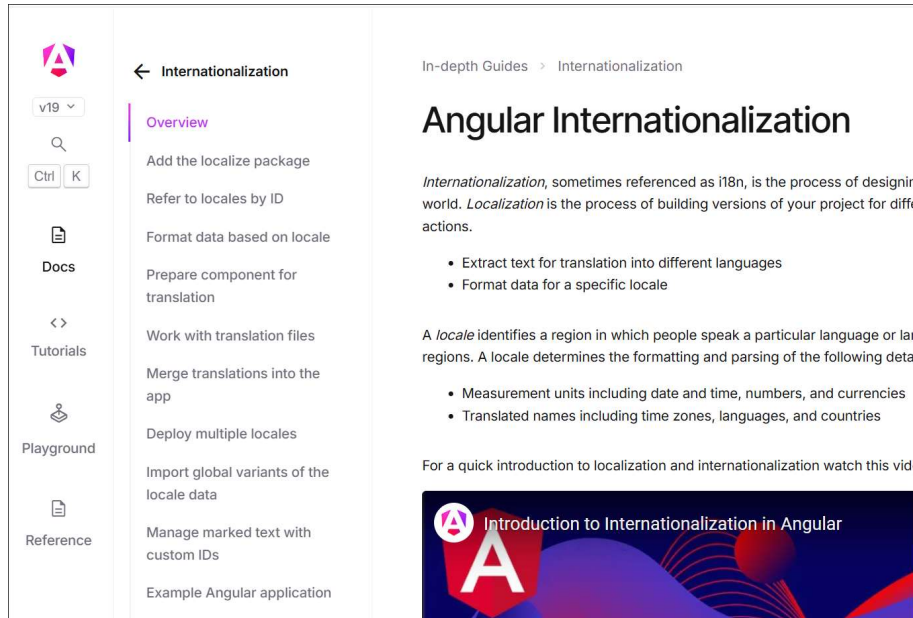
Internationalization

Using multiple languages and locales in your app

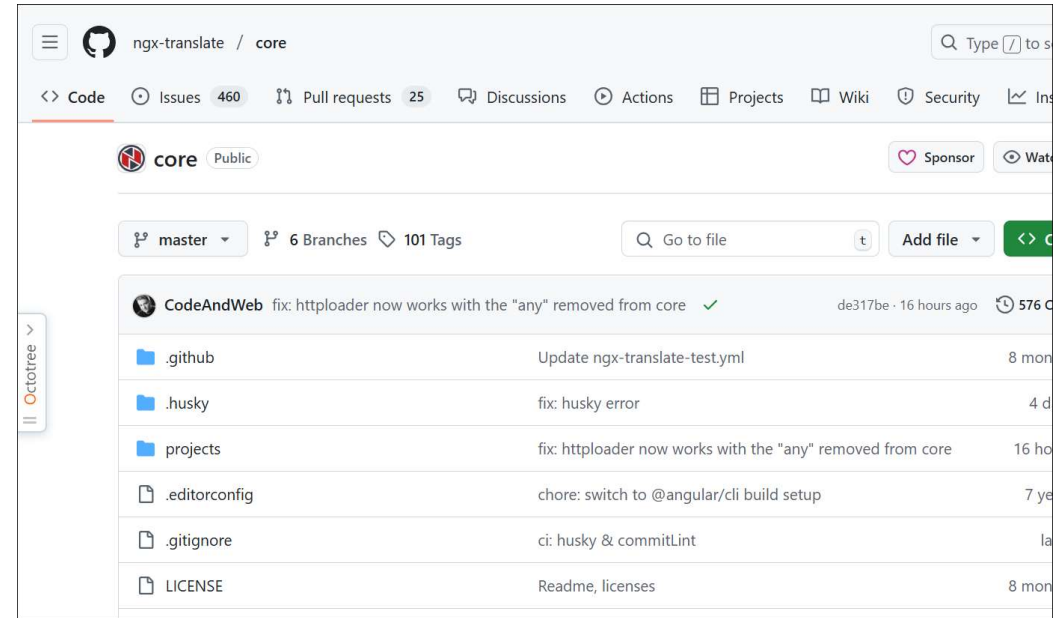
Options on translating

- **Default Angular** solution:
 - Translations are created compile time
 - So, 5 languages? 5 different versions of your app
 - Pro: very fast, good UI experience, more extensive (?)
 - Con: takes more disk space, involves extra compile step, takes reload to switch languages
 - <https://angular.dev/guide/i18n#>
- **External package**, `ngx-translate` (often regarded as 'default')
 - Usage of the `| translate` pipe, one (1) version of your app
 - *.json files with translations.
 - Pro: user can switch languages without a reload
 - Con: possible performance issues, when using (millions of) translations, since every binding needs to be re-evaluated on change detection cycle.
 - <https://github.com/ngx-translate/core>

Websites



<https://angular.dev/guide/i18n#>



<https://github.com/ngx-translate/core>

Modern solution: transloco

The screenshot shows the Transloco project page. At the top left is the Transloco logo (a blue square with a white 'T' and a blue square with a white 'L'). To the right is a 'Support' button with a heart icon and a 'More' dropdown. Below the logo is a version selector showing 'v8'. The main header area features the Transloco logo and a 'Copy' button. A message encourages support with a link to 'read here'. A row of badges shows 'npm v8.2.1', 'bundlephobia inaccessible', 'downloads 704k/month', 'CI passing', and 'PRs welcome'. A green box contains a message about contributions and feedback. The main text describes Transloco's functionality: allowing users to define translations in different languages and switch between them at runtime. It mentions a rich API and multiple plugins. Below this, it says 'Here is a small taste of the features it offers:' and lists two features: 1. Runtime Language Switching and 2. Clean and DRY Templates. The first feature describes dynamic language switching at runtime. The second feature describes a structural directive (*transloco) that facilitates efficient translations within templates, promoting a DRY approach by creating a single subscription per template.

Transloco

Want to support the project? [read here](#) for more information

npm v8.2.1 bundlephobia inaccessible downloads 704k/month CI passing PRs welcome

Help Transloco improve! Found an issue or have suggestions? I welcome [contributions](#) and [feedback](#) from the community.

Transloco allows you to define translations for your content in different languages and switch between them easily in runtime. It exposes a rich API to manage translations efficiently and cleanly. It provides multiple plugins that will improve your development experience.

Here is a small taste of the features it offers:

- Runtime Language Switching**

Transloco enables dynamic language switching at runtime, allowing users to change languages without reloading the application. This feature enhances user experience by providing seamless transitions between different languages.
- Clean and DRY Templates**

The library provides a structural directive (`*transloco`) that facilitates efficient translations directly within templates. This method promotes a DRY (Don't Repeat Yourself) approach by creating a single subscription per template, ensuring efficient change detection and updates.

<https://jsverse.gitbook.io/transloco>

Transloco features

- Runtime language switching
- Structural directive in templates (`*transloco`)
- Lazy loading by default
- Plugin ecosystem
- Modular

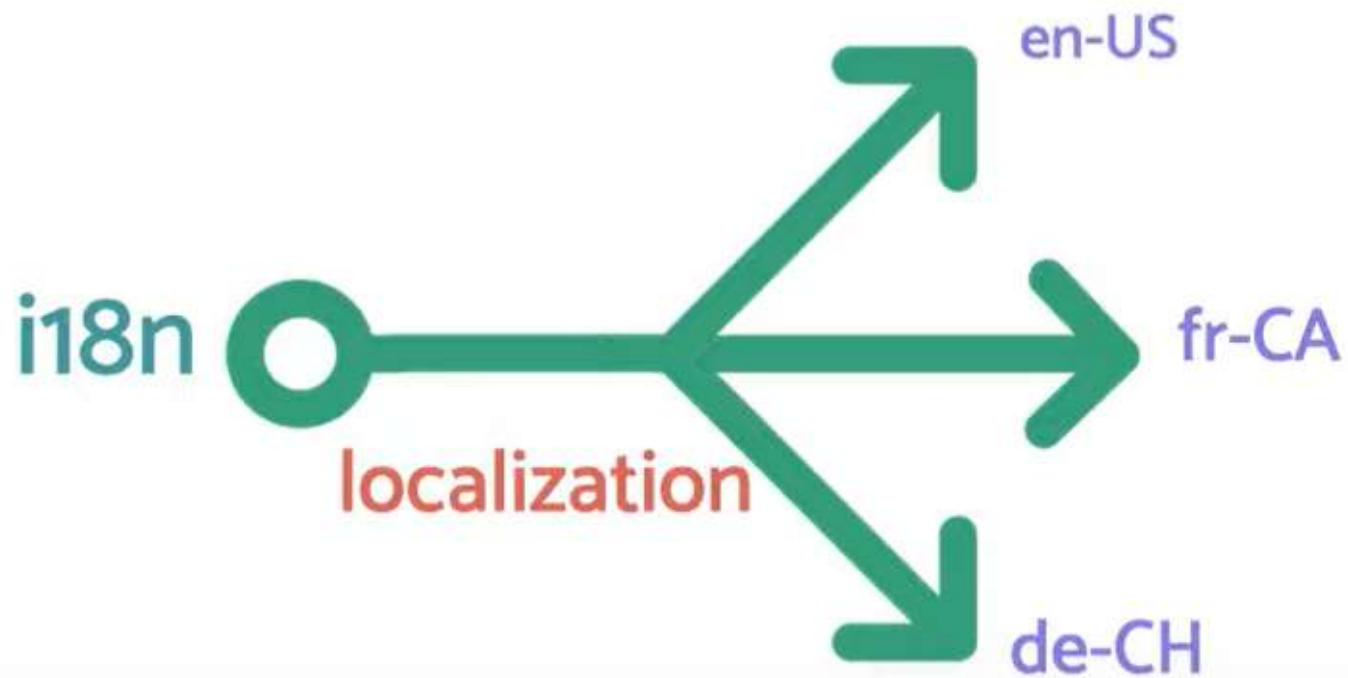
I18n Process

- Internationalization vs Localization
 - Related, but NOT the same!
- **Internationalization (i18n):**
 - Preparing your app to use multiple languages
 - Separating content for translation
 - Updating angular.json with locales
- **Localization:**
 - Building your app for a specific language
 - Default: NO RUNTIME SWITCHING of languages
 - Each locale gets its own build

Locales

- Are mentioned in the format `{language}-{locale}`
- For instance:
 - `en-US`
 - `fr-CH`
 - `nl-NL`
- Internationalization happens **ONCE (1x)**
- Localization happens **MANY TIMES** (as much locales as you want to support)
- You can change locale, without rewriting your application code
 - Page reload required. NOT at runtime with default Angular solution.

Internationalization



3 steps in localization

- 1. **Install** dependencies
 - Using `npm install <package>...`
- 2. **internationalize** the application
 - Add i18n tags and attributes to existing code
- 3. **Localize** the application
 - Repeat step 3 for every locale you want to add

A sample internationalization (i18n) app

Angular (also referred to as Angular 2+) is a TypeScript-based free and open-source single-page web application framework. It is developed by Google and a community of individuals and corporations. Angular is a complete rewrite from the same team that built AngularJS. The Angular ecosystem consists of a diverse group of library authors, and content creators. According to the Stack Overflow Developer Survey, Angular is one of the most commonly used web frameworks.

Source: wikipedia.org, [Angular](#)

Switch languages

French

English

.../735-ngx-translate

1. Add dependencies

- `ng add @angular/localize`
 - Reply `Yes` to confirm installation

```
ng add @angular/localize
  ã Determining Package Manager
    › Using package manager: npm
  ã Searching for compatible package version
    › Found compatible package version: @angular/localize@19.2.8.
  ã Loading package information from registry
  \ Confirming installation

The package @angular/localize@19.2.8 will be installed and executed.
Would you like to proceed? (Y/n)
```

When adding localization

- Files are updated:
 - `@angular/localize` added in `angular.json` as polyfill
 - `/// <reference types="@angular/localize" />` added to `main.ts`
 - `"types": [ "@angular/localize" ]` added to `tsconfig.json`

^ Loading package information from registry

Ã Confirming installation

Ã Installing package

UPDATE src/main.ts (301 bytes)

UPDATE tsconfig.app.json (472 bytes)

UPDATE tsconfig.spec.json (477 bytes)

UPDATE angular.json (2834 bytes)



2. Preparing your app for i18n

- Add the `i18n` attribute to tags/content you want to translate
- Simple attribute for default translation, more finegrained options available for adding custom `id's`, `meaning`, `description`.

```
<div class="container">  
  <h1 i18n>A sample internationalization (i18n) app</h1>  
  <hr>  
  <p i18n>  
    Angular (also referred to as Angular 2+) is a ...  
    ...  
  </p>  
</div>
```



3. Extract elements

- Extract all elements that has been marked for translation
 - using the `extract-i18n` tool
- 1: create a `src/locale` directory.
 - This will contain the translation files
- 2: `ng extract-i18n --output-path src/locale`
 - This will create a `messages.xlf` file in the `src/locale` directory.

```
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <xliff version="1.2" xmlns="urn:oasis:names:tc:xliff:document:1.2">
3   <file source-language="en-US" datatype="plaintext" original="ng2.template">
4     <body>
5       <trans-unit id="8289536221293264814" datatype="html">
6         <source>A sample internationalization (i18n) app</source>
7         <context-group purpose="location">
8           <context context-type="sourcefile">src/app/app.component.html</context>
9           <context context-type="linenumber">2,3</context>
10        </context-group>
11      </trans-unit>
12      <trans-unit id="3758043197462175605" datatype="html">
```

4. Add other translations

- `.xlf` is a standard format, used by translation agencies worldwide
 - <https://en.wikipedia.org/wiki/XLIFF>
- Copy the `messages.xlf` file to (for instance) `messages.fr.xlf` for French translations
 - Copy and rename to cover other languages.
- Cover all messages and translate them
 - Often professional translation software that can read/write `.xlf` is used.
 - Not recommended to do this directly in the XML.

```
<xliff version="1.2" xmlns="urn:oasis:names:tc:xliff:document:1.2">
  <file source-language="en" target-language="fr" datatype="plaintext" original="ng2">
    <body>
      <trans-unit id="1234567890" datatype="html">
        <source>A sample internationalization (i18n) app</source>
        <target>Une application d'internationalisation (i18n) simple</target>
      </trans-unit>
      <trans-unit id="2345678901" datatype="html">
        <source>Angular (also referred to as Angular 2+) is a TypeScript-based free
        <target>Angular (également appelé Angular 2+) est un framework d'application
```

5. Update angular.json to support .fr

- Update the section `projects.<name>.architect.build.options`
- Documentation: <https://angular.dev/guide/i18n/merge#angularjson-for-en-us-and-fr-example>

For example, the following excerpt of an `angular.json` workspace build configuration file sets the source locale to `en-US` and provides the path to the French (`fr`) locale translation file.

angular.json

```
"projects": {  
  "angular.io-example": {  
    ...  
    "i18n": {  
      "sourceLocale": "en-US",  
      "locales": {  
        "fr": {  
          "translation": "src/locale/messages.fr.xlf",  
          ...  
        }  
      }  
    }  
  }  
}
```

6. Deploy the locales

- Build application. If you use the flag `--localize`, all locales are generated.
 - `ng build --localize`
 - <https://angular.dev/guide/i18n/deploy>

The screenshot shows the Angular documentation page for 'Deploy multiple locales'. On the left is a sidebar with a list of topics under the 'Internationalization' heading. The main content area has a breadcrumb 'In-depth Guides > Internationalization' and a title 'Deploy multiple locales'. The text explains that if `myapp` is the project directory, different versions for different locales are placed in subdirectories like `myapp/fr` and `myapp/es`. It also describes how the `base` and `href` attributes in HTML tags are adjusted for each locale based on the `angular.json` configuration. A code block shows the relevant part of the `angular.json` file, which includes a `projects` object with an `angular.io-example` entry.

← Internationalization

- Overview
- Add the localize package
- Refer to locales by ID
- Format data based on locale
- Prepare component for translation
- Work with translation files
- Merge translations into the app
- Deploy multiple locales**
- Import global variants of the locale data
- Manage marked text with custom IDs

In-depth Guides > Internationalization

Deploy multiple locales

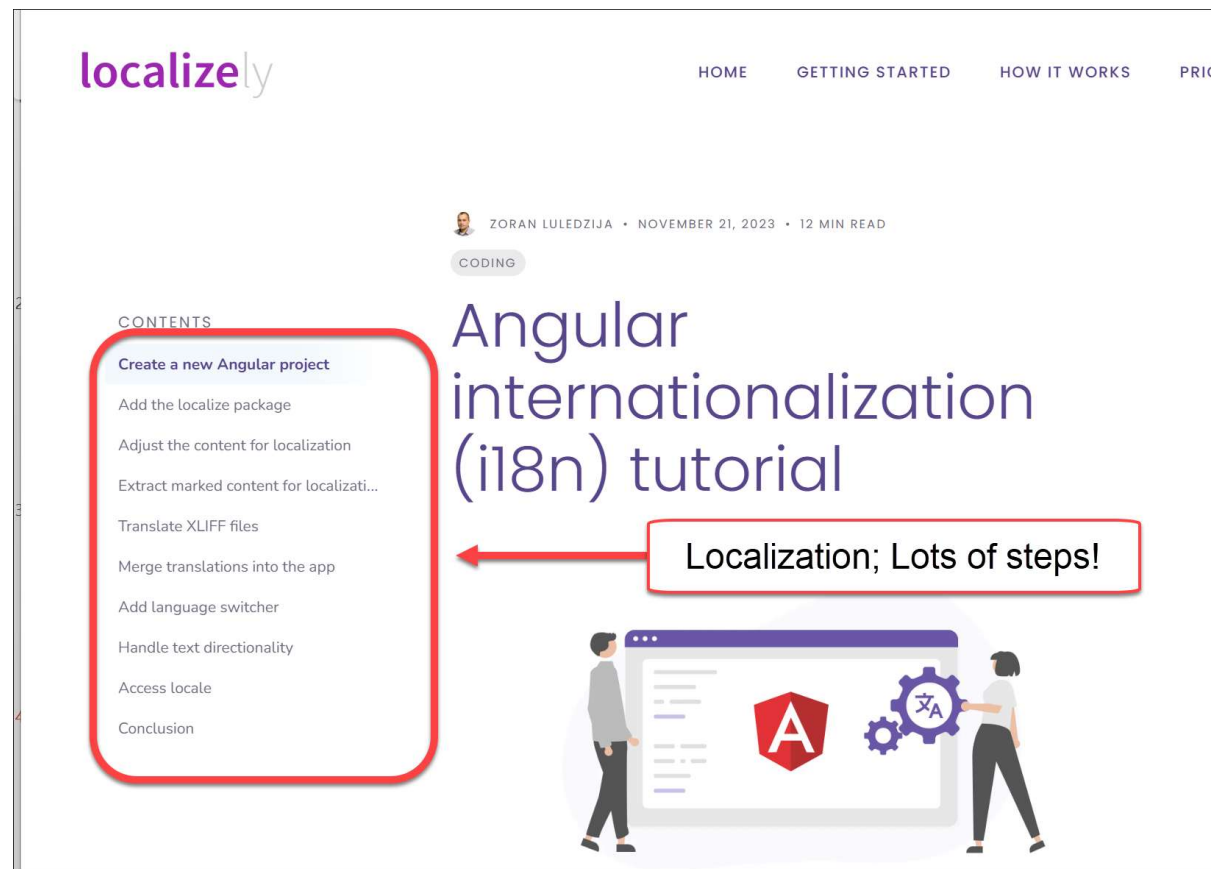
If `myapp` is the directory that contains the distributable files of your project, you typically make different versions available for different locales in locale directories. For example, your French version is located in the `myapp/fr` directory and the Spanish version is located in the `myapp/es` directory.

The HTML `base` tag with the `href` attribute specifies the base URI, or URL, for relative links. If you set the `"localize"` option in `angular.json` workspace build configuration file to `true` or to an array of locale IDs, the CLI adjusts the base `href` for each version of the application. To adjust the base `href` for each version of the application, the CLI adds the locale to the configured `"subPath"`. Specify the `"subPath"` for each locale in your `angular.json` workspace build configuration file. The following example displays `"subPath"` set to an empty string.

```
angular.json
{
  "projects": {
    "angular.io-example": {
      ...
    }
  }
}
```

Read Article

- Refer to official documentation
- Use article from localizely: <https://localizely.com/blog/angular-i18n-tutorial/>

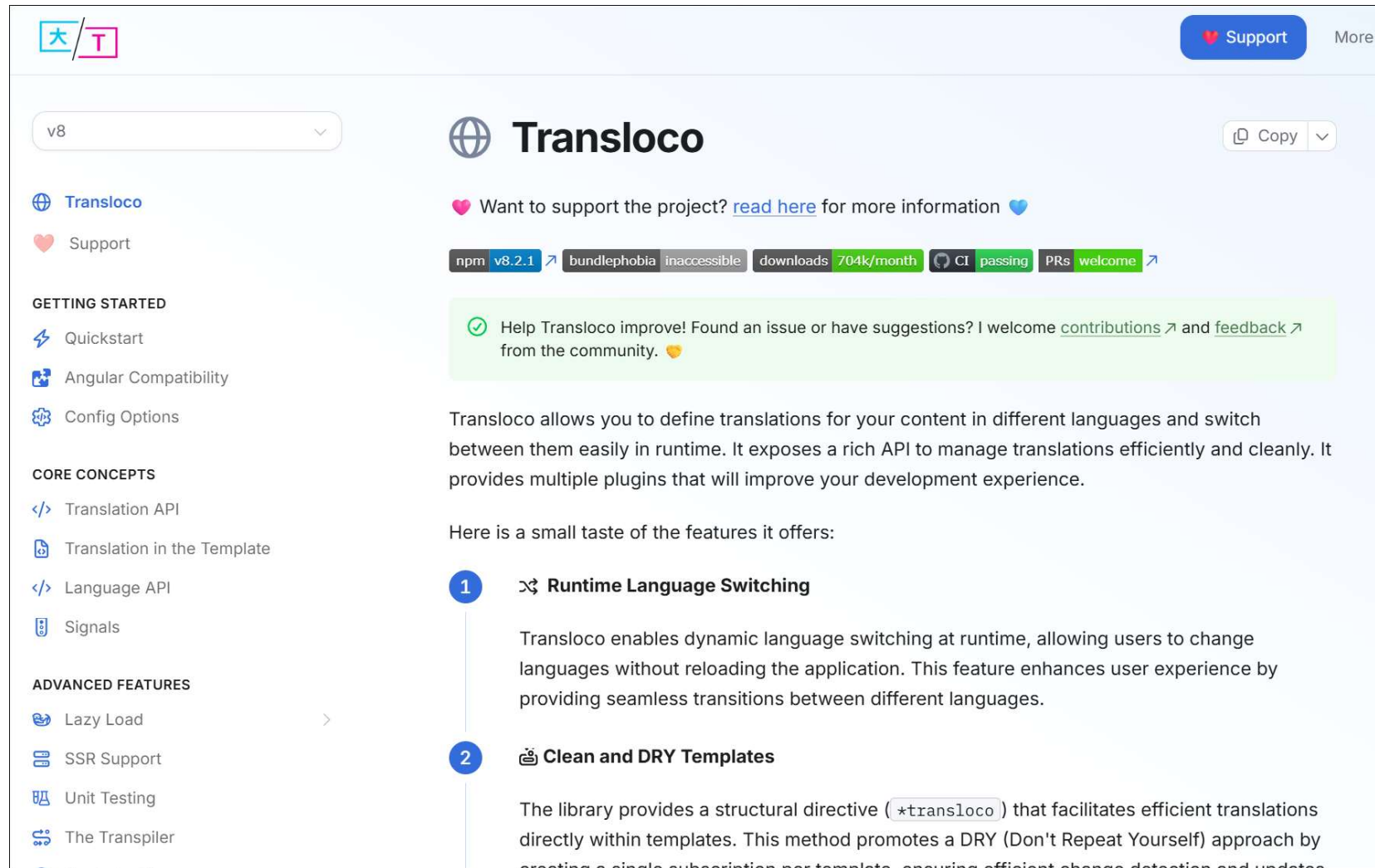




Transloco

An i18n solution for modern Angular applications

Homepage Transloco project



The screenshot shows the homepage of the Transloco project. The header includes a logo with a blue '大' and a pink 'T', a 'Support' button with a heart icon, and a 'More' link. Below the header, there's a version selector set to 'v8'. The main heading is 'Transloco' with a globe icon. A call to action asks if the user wants to support the project, with a link to 'read here'. A status bar shows 'npm v8.2.1', 'bundlephobia', 'inaccessible', 'downloads 704k/month', 'CI passing', and 'PRs welcome'. A green box contains a message encouraging contributions and feedback. The main text describes Transloco as a library for defining translations in different languages and switching between them at runtime. It lists features: 1. Runtime Language Switching and 2. Clean and DRY Templates. The left sidebar has links for 'Transloco', 'Support', 'GETTING STARTED' (Quickstart, Angular Compatibility, Config Options), 'CORE CONCEPTS' (Translation API, Translation in the Template, Language API, Signals), and 'ADVANCED FEATURES' (Lazy Load, SSR Support, Unit Testing, The Transpiler).

大/T

Support

More

v8

Transloco

Want to support the project? [read here](#) for more information

npm v8.2.1 bundlephobia inaccessible downloads 704k/month CI passing PRs welcome

Help Transloco improve! Found an issue or have suggestions? I welcome [contributions](#) and [feedback](#) from the community.

Transloco allows you to define translations for your content in different languages and switch between them easily in runtime. It exposes a rich API to manage translations efficiently and cleanly. It provides multiple plugins that will improve your development experience.

Here is a small taste of the features it offers:

- Runtime Language Switching**

Transloco enables dynamic language switching at runtime, allowing users to change languages without reloading the application. This feature enhances user experience by providing seamless transitions between different languages.
- Clean and DRY Templates**

The library provides a structural directive (`*transloco`) that facilitates efficient translations directly within templates. This method promotes a DRY (Don't Repeat Yourself) approach by creating a single subscription per template, ensuring efficient change detection and updates.

Transloco

Support

GETTING STARTED

- Quickstart
- Angular Compatibility
- Config Options

CORE CONCEPTS

- Translation API
- Translation in the Template
- Language API
- Signals

ADVANCED FEATURES

- Lazy Load
- SSR Support
- Unit Testing
- The Transpiler

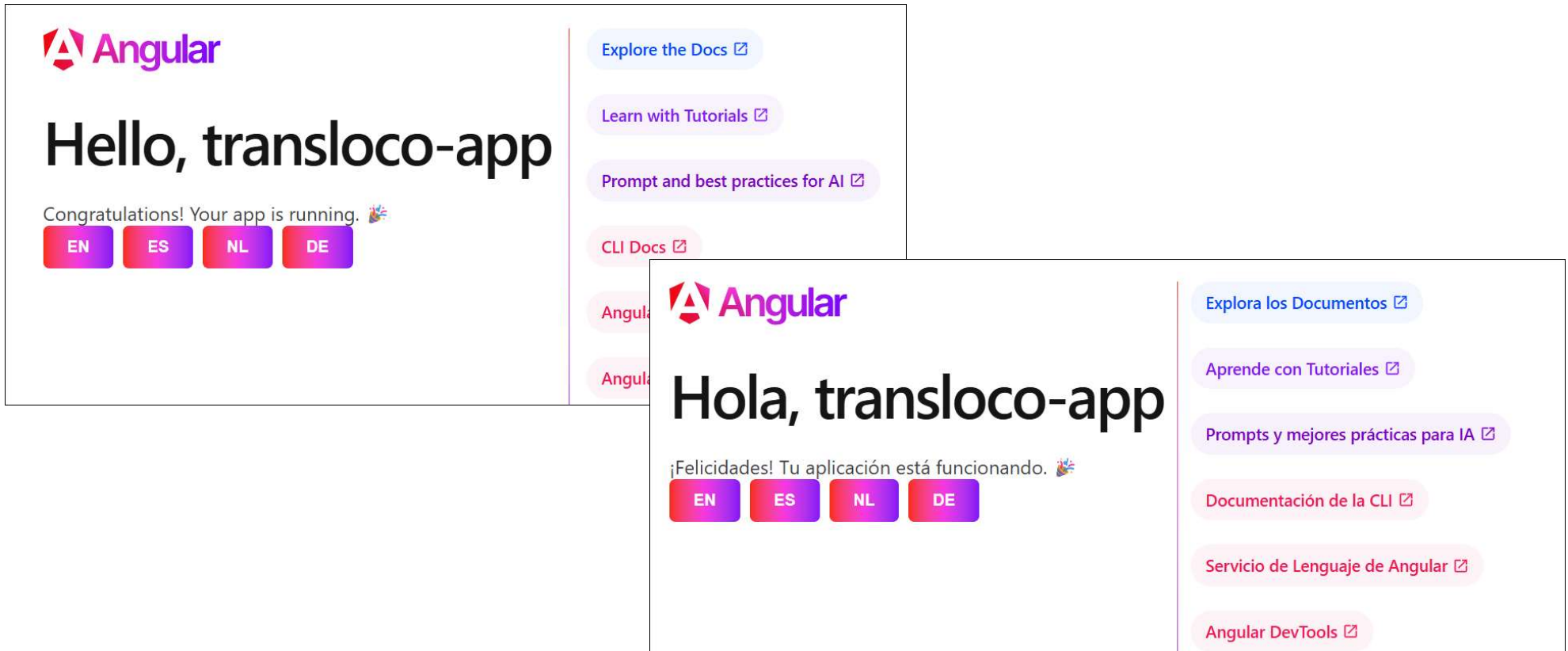
<https://jsverse.gitbook.io/transloco>

Creating a default Transloco app – Step #1

- Generate a new Angular app using the CLI (`ng new <appName>`)
- `ng add @jsverse/transloco`
- Some files are changed/enhanced:
 - `app.config.ts` → add the `provideTransloco` and `provideHttpClient` to app provider
 - `transloco.loader.ts` → create a default http loader implementation to fetch the translation files
 - `*.json` files (`public/en.json` | `es.json` | `nl.json`, etc) → json files containing objects with key/value-pairs
 - `transloco.config.ts` → transloco global config file.

Transloco next steps

- Now go on and translate the template, using `keys` instead of real values.
 - Don't forget to add `*transloco` directive and update the other files!



Transloco steps:

1. Update `*.json` (in this case: `en.json` and `es.json`) with translation strings.
2. Modify `app.ts` to include `TranslocoDirective`, `TranslocoService`, and a `toggleLanguage()` method.
3. Modify `app.html` to use the `*transloco` directive and translate the strings.
4. Add a language switcher button in the template, or use dropdown with flags, etc. Call `toggleLanguage()` from the switcher.
5. Verify `app.config.ts` has `reRenderOnLangChange: true` and both `en` and `es` (and possibly others) in `availableLangs`.

App.ts

```
export class App {  
  // TranslocoService is injected to handle language switching logic  
  private translocoService = inject(TranslocoService);  
  protected readonly title = signal('transloco-app');  
  
  // Set the application language to the specified language code  
  setLanguage(lang: string) {  
    this.translocoService.setActiveLang(lang);  
  }  
}
```

App.html

```
<!-- Use transloco directive to handle translations  
in this scope -->  
<main class="main" *transloco="let t">  
  <div class="content">  
    ...
```

```
<h1>{{ t('hello', {title: title()}) }}</h1>
```

```
@for (item of [  
  {title: t('links.explore'), link: 'https://angular.dev'},  
  {title: t('links.tutorials'), link: 'https://angular.dev/tutorials'},  
  ...  
]; track item.title) {
```

```
<button class="lang-switcher"  
(click)="setLanguage('en')">EN</button>
```

JSON file(s)

```
{  
  "hello": "Hello, {{title}}",  
  "congrats": "Congratulations! Your app is running. 🎉",  
  "links": {  
    "explore": "Explore the Docs",  
    "tutorials": "Learn with Tutorials",  
    "ai": "Prompt and best practices for AI",  
    "cli": "CLI Docs",  
    "language": "Angular Language Service",  
    "devtools": "Angular DevTools"  
  }  
}
```

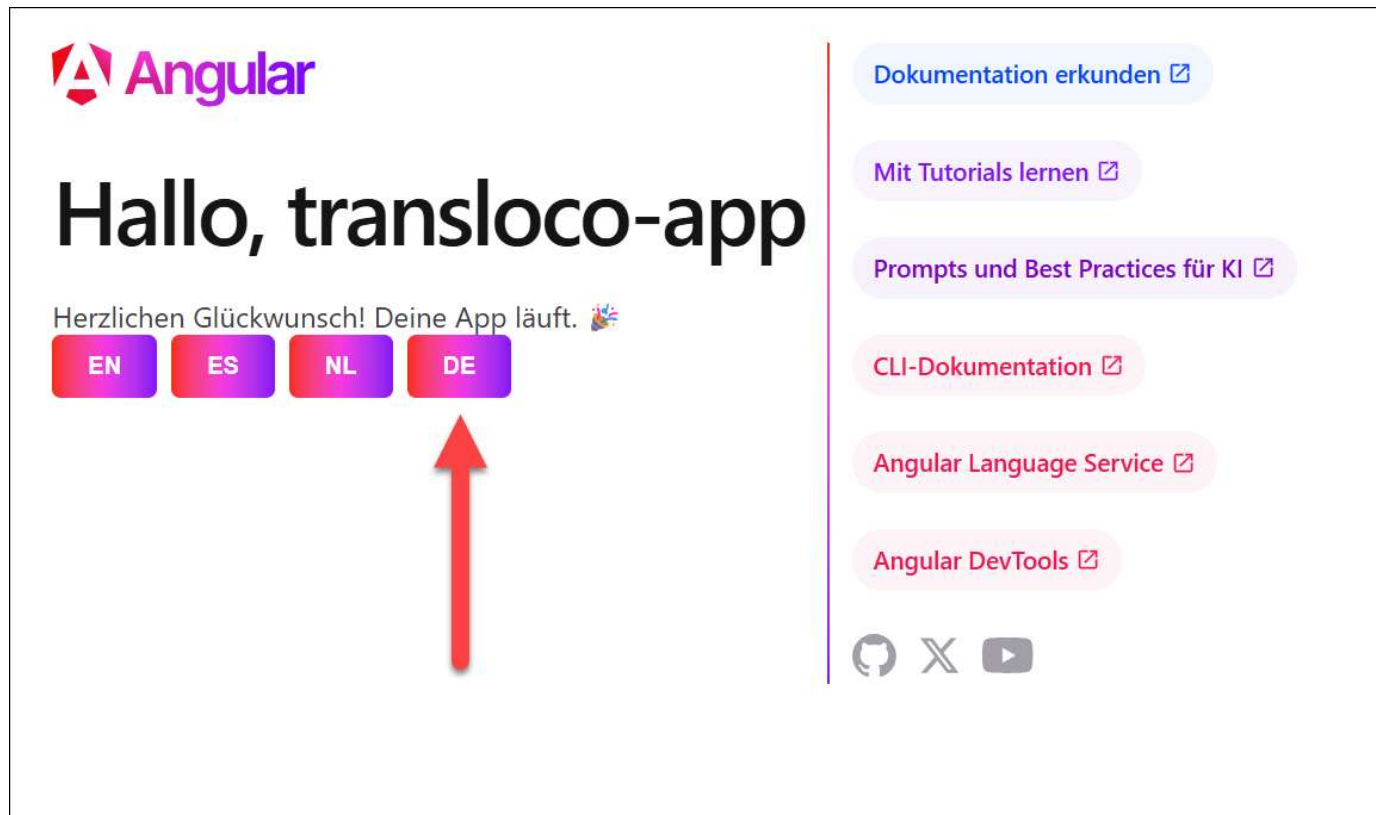
```
{  
  "hello": "Hola, {{title}}",  
  "congrats": "¡Felicidades! Tu aplicación está funcionando. 🎉",  
  "links": {  
    "explore": "Explora los Documentos",  
    "tutorials": "Aprende con Tutoriales",  
    "ai": "Prompts y mejores prácticas para IA",  
    "cli": "Documentación de la CLI",  
    "language": "Servicio de Lenguaje de Angular",  
    "devtools": "Angular DevTools"  
  }  
}
```

App.config.ts

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(),
    provideTransloco({
      config: {
        availableLangs: ['en', 'es', 'nl', 'de'],
        defaultLang: 'en',
        reRenderOnLangChange: true,
        prodMode: !isDevMode(),
      },
      loader: TranslocoHttpLoader
    })
  ]
};
```

Example:

- See Angular Advanced / 370-transloco-i18n



Verdict: ngx-translate vs transloco

- **Existing apps** with ngx-translate: if this setup is already running, stable, and to satisfaction, there are **little wins refactoring an app** to transloco.
 - *It is 'nice to have' instead of necessary*
- **New Apps**: transloco *might* be a good solution.
 - Better alignment with **modern Angular** (standalone config, provider-based setup, functional patterns).
 - Better support for **Scoped / feature translations** (Transloco supports organizing translations by feature/scope more naturally)
 - Better support for **lazy loading strategies** and better(?) performance reported
 - Reportedly **better Developer Experience (DX)** – though this might be personal and biased